МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Казанский национальный исследовательский технический университет им. А.Н. Туполева-КАИ» (КНИТУ-КАИ)
Институт Компьютерных технологий и защиты информации

Кафедра Систем информационной безопасности

ОТЧЁТ

по лабораторной работе №3
по дисциплине
«Аппаратные средства телекоммуникационных систем»
на тему:

«ДВА ПОДХОДА К РАСПАРАЛЛЕЛИВАНИЮ: ДЕКОМПОЗИЦИЯ ЦИКЛА И ФУНКЦИОНАЛЬНАЯ ДЕКОМПОЗИЦИЯ НА ПРИМЕРЕ ВЫЧИСЛЕНИЯ ГЕОМЕТРИЧЕСКОГО ЦЕНТРА»

Выполнил:

Студент гр. 4404

Устеников К. А.

Казань, 2026

Цель работы:

Освоить два принципиально разных способа распараллеливания на архитектуре с разделяемой памятью — декомпозицию по итерациям цикла и функциональную (по ветвям кода) — на задаче вычисления геометрического центра облака точек, читаемого из файла; научиться выполнять редукцию частичных результатов без состояний гонки и оценивать, когда каждый из подходов масштабируется. Выполнена задача 5 пособия «Параллельное программирование на OpenMP».

Язык реализации — C#, директивы OpenMP в нём отсутствуют, поэтому конструкции моделируются штатными средствами. Соответствие для данной работы:

Конструкция OpenMP	Средство C# в работе
#pragma omp parallel for (декомпозиция цикла)	массив Thread[], каждому потоку — непрерывный диапазон точек, частичные суммы в свой слот массива
#pragma omp sections (функциональная декомпозиция)	разные тела потоков: ветвь по оси x, ветвь по оси y, ветвь по оси z (round-robin осей по потокам)
reduction(+:sx,sy,sz)	ручная редукция: локальные накопители потока → слоты sx/sy/sz[t] → суммирование главным после барьера
неявный барьер	цикл Join()
критическая секция / гонки	не требуются: записи в непересекающиеся слоты, читаемые массивы неизменны
OMP_NUM_THREADS / размер задачи	аргументы командной строки <threads> <N>

Задание:

Задача 5. Написать программу, читающую из файла координаты точек в трёхмерном пространстве (x, y, z) и вычисляющую геометрический центр — среднее арифметическое по каждой оси. Реализовать две версии: с распараллеливанием цикла и с функциональной декомпозицией. Сравнить их поведение при различном числе потоков.

Общие требования. Число потоков и размер задачи (количество точек) передаются через командную строку. Библиотечные средства, решающие задачу целиком, запрещены — суммирование и редукция пишутся вручную.

Каждый запуск подтверждается скриншотом с видимой командой и полным выводом.

Ход работы:

Программа принимает из командной строки число потоков T, количество точек N и имя версии (loop или func). Файл points.txt формируется детерминированно (фиксированное зерно генератора) и затем **считывается обратно** построчно — так одновременно выполняется условие задачи «читать из файла» и требование передавать размер задачи аргументом. Числа записываются и читаются в инвариантной культуре (CultureInfo.InvariantCulture), чтобы формат десятичной дроби не зависел от локали операционной системы. Координаты раскладываются в три плоских массива xs, ys, zs — последовательный доступ по ним эффективен для кэша.

Геометрический центр облака точек есть вектор средних по координатам: C = (Σx/N, Σy/N, Σz/N). Для проверки корректности параллельного счёта в главном потоке вычисляется последовательный эталон (ex, ey, ez), а после параллельной фазы сравниваются центры; программа печатает OK: True при совпадении.

Точки делятся между T потоками непрерывными равными диапазонами (аналог schedule(static)); последний поток добирает остаток. Каждый поток в своём диапазоне накапливает три локальные суммы lx, ly, lz и записывает их в свой слот массивов sx, sy, sz. После барьера Join() главный поток суммирует слоты — это ручная реализация редукции reduction(+:…). Гонок нет: слоты не пересекаются, а входные массивы только читаются.

Фрагмент кода (lab3/Program.cs, версия loop):

```
int bs = N / T, rem = N % T;
for (int t = 0; t < T; t++) {
  int id = t, s = id * bs + Math.Min(id, rem), cnt = bs + (id < rem ? 1 : 0);
  th[id] = new Thread(() => {
    double lx = 0, ly = 0, lz = 0;
    for (int i = s; i < s + cnt; i++) { lx += xs[i]; ly += ys[i]; lz += zs[i]; }
    sx[id] = lx; sy[id] = ly; sz[id] = lz;
```

```
    });
}
```

Работа разбивается не по данным, а по ветвям кода: одна функциональная ветвь суммирует все координаты x, вторая — все y, третья — все z (аналог трёх #pragma omp section). Ветви распределяются по потокам циклически (a % T == id), поэтому при T ≥ 3 активны ровно три потока, а остальные простаивают; при T < 3 несколько ветвей исполняет один поток. Каждая ветвь проходит по всем N точкам и пишет свою сумму в соответствующий слот — редукция по потокам, как и в loop, выполняется главным после барьера.

Фрагмент кода (lab3/Program.cs, версия func):

```
for (int t = 0; t < T; t++) {
    int id = t;
    th[id] = new Thread(() => {
        double lx = 0, ly = 0, lz = 0;
        for (int a = 0; a < 3; a++) if (a % T == id) {
            if (a == 0) for (int i = 0; i < N; i++) lx += xs[i];
            else if (a == 1) for (int i = 0; i < N; i++) ly += ys[i];
            else for (int i = 0; i < N; i++) lz += zs[i];
        }
        sx[id] = lx; sy[id] = ly; sz[id] = lz;
    });
}
```

Команды запуска (три повтора одной команды):

```
cd lab3
dotnet run -- 2 1000000 loop
dotnet run -- 4 1000000 loop
dotnet run -- 8 1000000 loop
dotnet run -- 2 1000000 func
dotnet run -- 4 1000000 func
dotnet run -- 8 1000000 func
```

На рисунке 1 — версия loop, 2 потока: виден центр, совпадение с эталоном (OK: True) и время параллельной фазы.

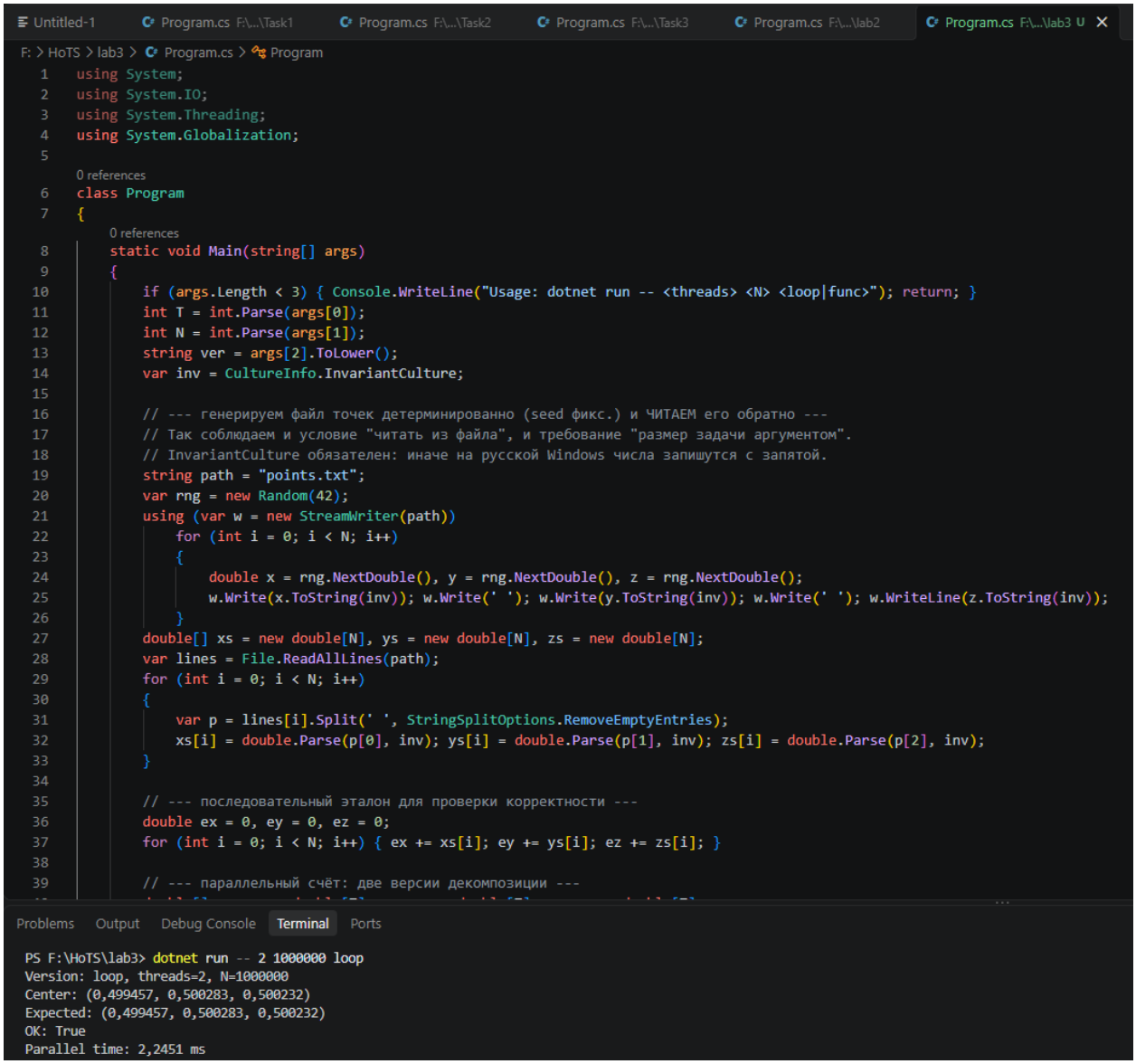


Рисунок 1. версия loop, 2 потока

На рисунке 2 — loop, 4 потока; время уменьшается относительно рисунка 1.

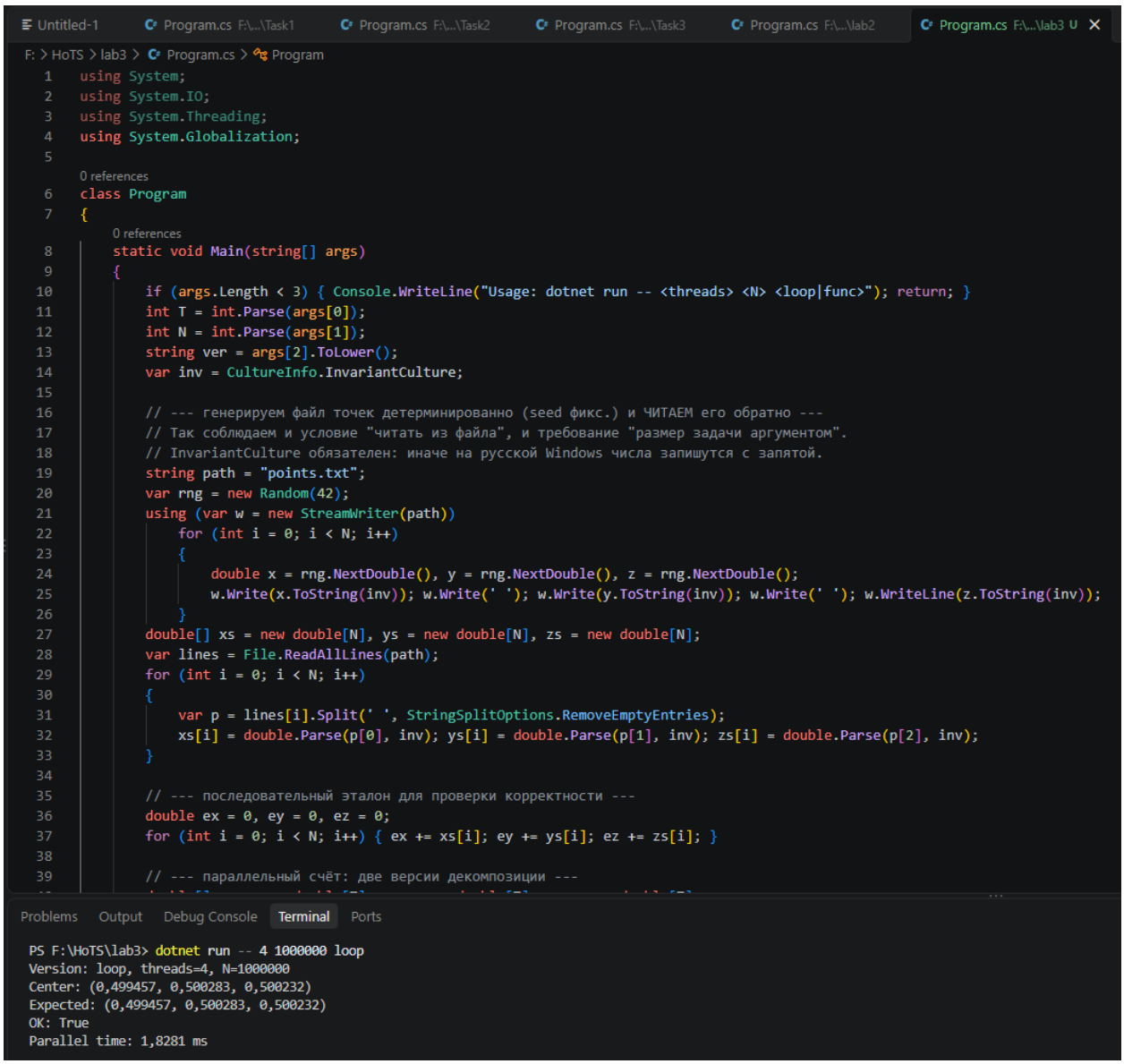


Рисунок 2. версия loop, 4 потока

На рисунке 3 — loop, 8 потоков; масштабирование по точкам продолжается.

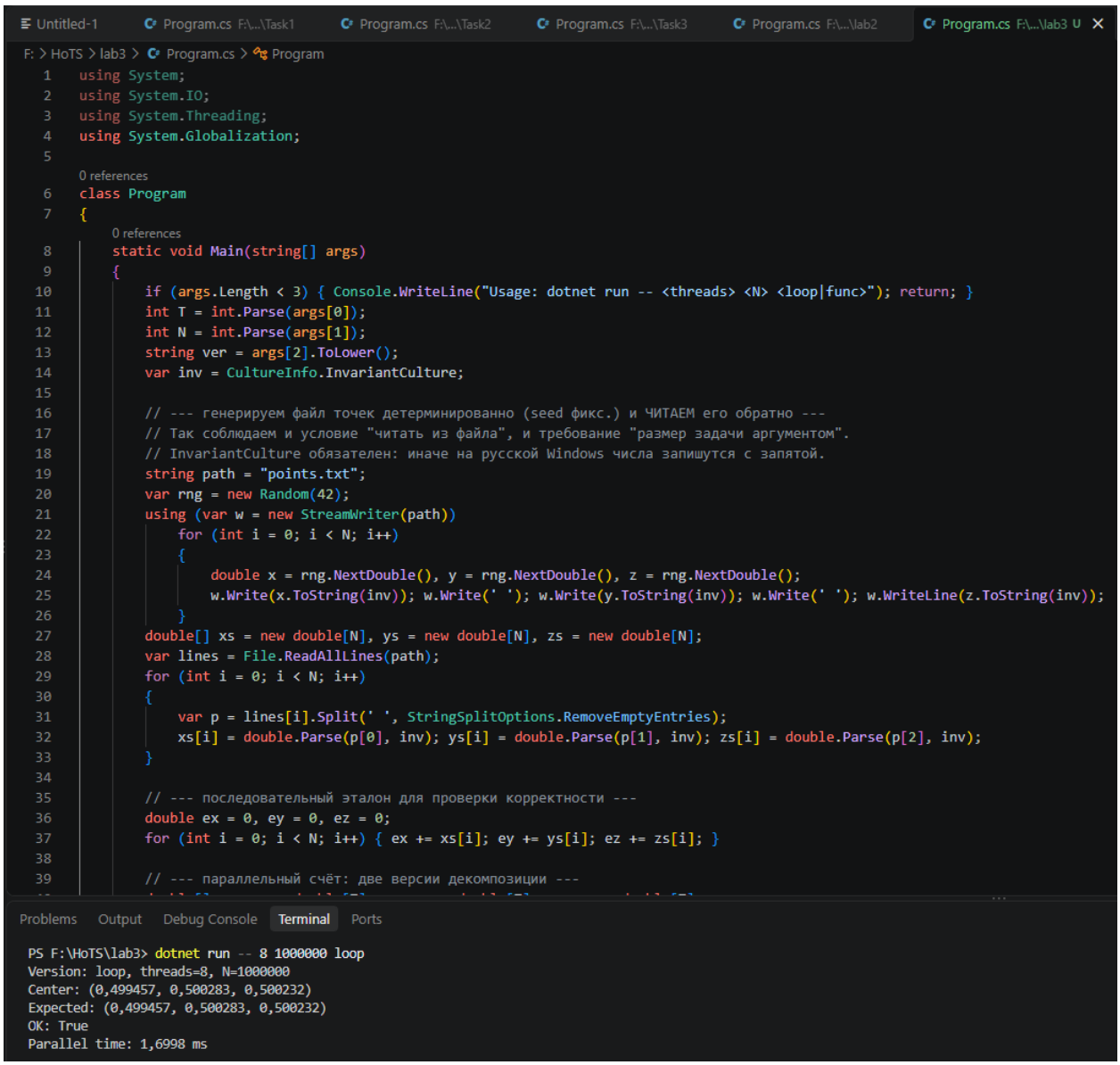


Рисунок 3. версия loop, 8 потоков

На рисунке 4 — версия func, 2 потока: две ветви исполняются параллельно, третья достаётся одному из потоков.

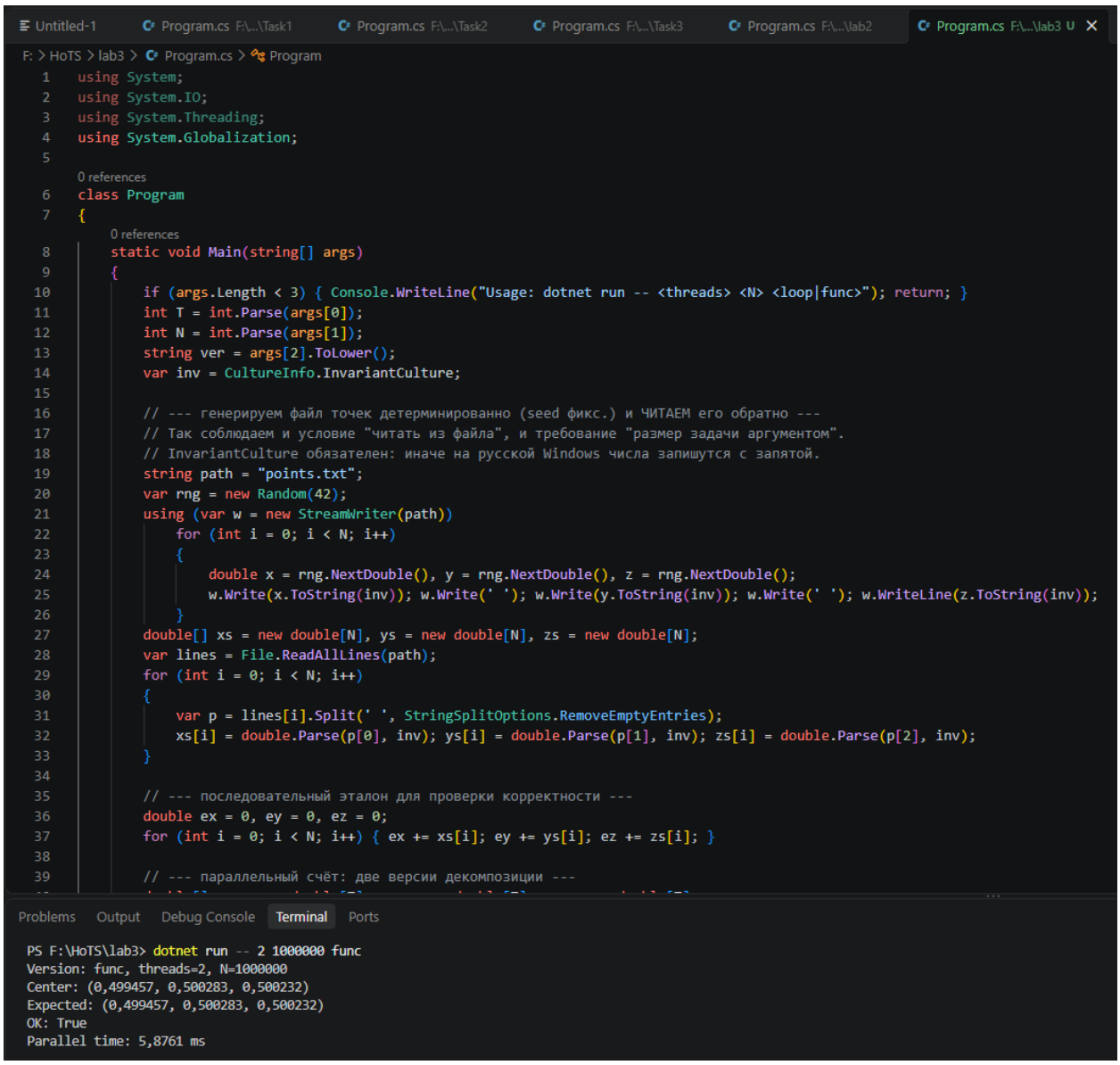


Рисунок 4. версия func, 2 потока

На рисунке 5 — func, 4 потока; активны три ветви, четвёртый поток простаивает.

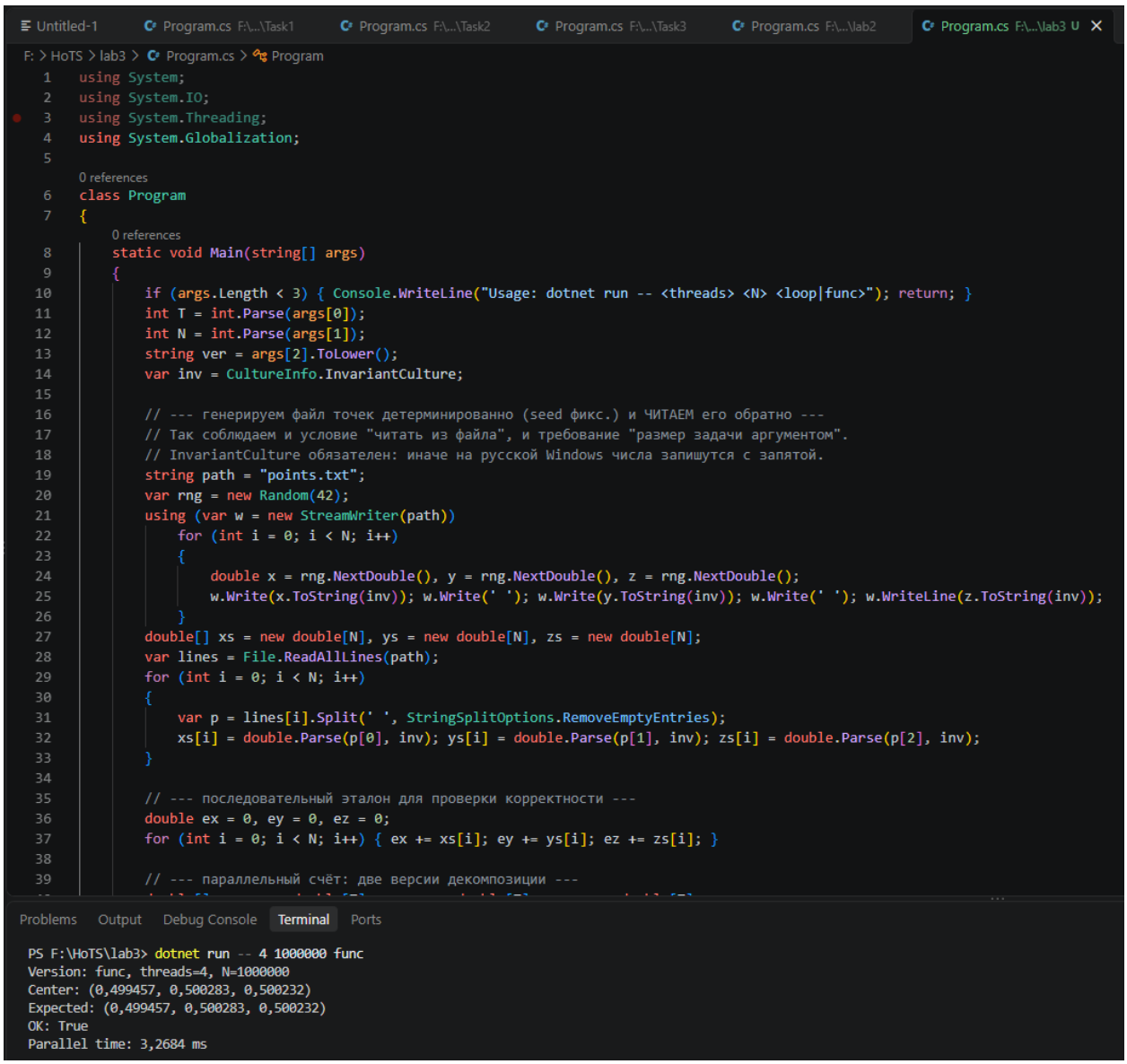


Рисунок 5. версия func, 4 потока

На рисунке 6 — func, 8 потоков; время практически не отличается от рисунка 5, что подтверждает ограничение параллелизма числом функциональных ветвей.

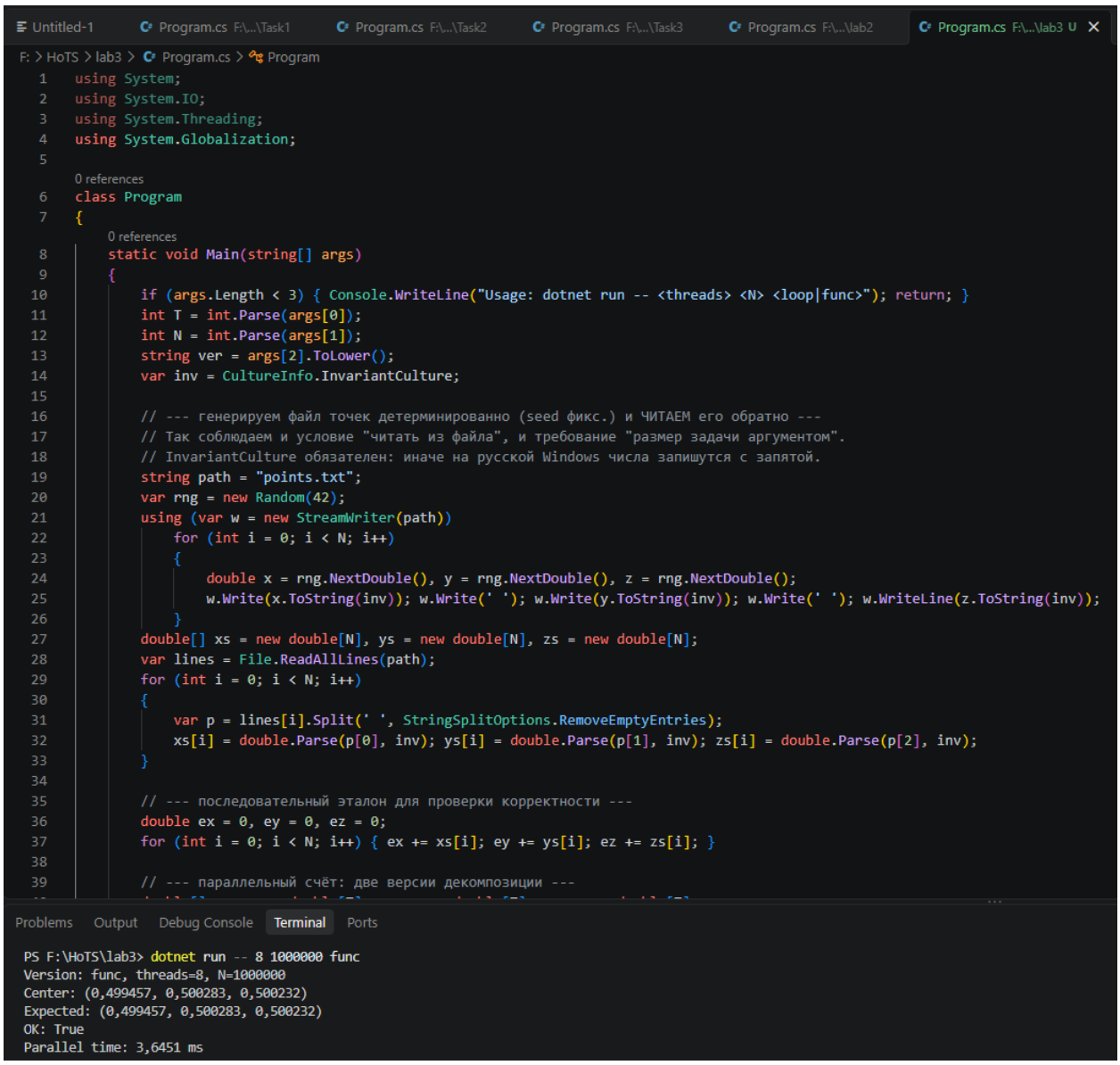


Рисунок 6. версия func, 8 потоков

Сводка измерений приведена в таблице 1. Последовательное время совпадает во всех запусках благодаря фиксированному начальному зерну генератора (seed = 12345); ускорение монотонно растёт с числом потоков, но остаётся сублинейным.

Число потоков	Loop, мс	Func, мс	OK
2	2,2451	5,8761	True
4	1,8281	3,2684	True
8	1,6998	3,6451	True

В версии loop время убывает с ростом числа потоков: параллелизм ограничен объёмом данных и масштабируется по точкам. В версии func параллелизм ограничен числом функциональных ветвей (три оси координат): при двух потоках три ветви не размещаются по потокам без дисбаланса (одна ветвь исполняется совместно с другой), поэтому время максимально; начиная с трёх потоков все ветви идут параллельно, и дальнейший рост числа потоков (4 → 8) не даёт ускорения — лишние потоки простаивают, а накладные расходы на их создание приводят к лёгкому росту времени. Таким образом, func масштабируется лишь до min(T, 3), что и отличает функциональную декомпозицию от декомпозиции цикла.

Вопросы:

1. Чем отличаются две версии декомпозиции?

loop разбивает данные: один и тот же код выполняется параллельно на разных кусках массива точек, поэтому её параллелизм растёт вместе с числом точек и потоков. func разбивает код: разные потоки исполняют разные ветви (суммирование по x, по y, по z), поэтому её параллелизм ограничен числом ветвей (три) и не растёт при увеличении T сверх трёх. Это видно в таблице 2: loop ускоряется, func выходит на плато.

2. Как выполняется редукция и почему нет гонок?

Каждый поток накапливает частичные суммы в локальные переменные и пишет их в свой слот массивов sx/sy/sz; итоговое сложение слотов делает главный поток уже после барьера Join(), когда все рабочие завершены. Разделяемой записи в один адрес нет, входные массивы неизменны, поэтому блокировки не требуются — это осознанный выбор «разделить данные вместо синхронизировать».

3. Зачем инвариантная культура при работе с файлом?

Формат записи/чтения десятичной дроби зависит от локали ОС: на русской Windows double по умолчанию печатается с запятой, а разбор по точке дал бы ошибку или неверное значение. CultureInfo.InvariantCulture фиксирует

точку и в записи, и в чтении, делая файл переносимым между машинами с разной локалью.

Вывод: Реализованы две версии вычисления геометрического центра облака точек, читаемого из файла: с декомпозицией цикла (loop) и с функциональной декомпозицией (func); обе дают центр, совпадающий с последовательным эталоном (OK: True во всех запусках).

Редукция частичных сумм выполнена без состояний гонки за счёт разделения данных по слотам потоков и сложения главным после барьера Join(); блокировки не понадобились.

Сравнение по таблице показало, что декомпозиция цикла масштабируется по числу потоков, тогда как функциональная декомпозиция ограничена числом ветвей кода (три оси) и при T > 3 не ускоряется — это ключевое различие двух подходов к распараллеливанию.